

Assignment: 01

Due: Tuesday, September 22, 2026 9:00 pm

Coverage: L02

Language level: Beginning Student

Files to submit: `functions.rkt`

Assignment policies for this and all subsequent assignments

- The deadline for submission without penalty is stated above. More details are listed on [the course website's assignment policies page](#).
- The work you submit must be entirely your own.
- Do not look up or generate either full or partial solutions on the web or using AI tools.
- Please comment your submissions as described in the lecture notes. Please read the course Web page for more information on how to submit your work.

Grading

- **You will not receive marks for A01 unless you have earned full marks for A00 before the A01 due date.**
- Assignments may be submitted as often as desired up to the due date. Your grade for functions you write will be based on your last successful submission.
- For this assignment only, your grade for the functions you write is based solely on *correctness*, i.e., does your implementation produce the correct value for the arguments it consumes?
- When you submit your assignment, you will receive the results of *basic tests*.
- **Be sure to check your basic test results after each submission!** Your basic test results will be available shortly after you make a submission. Check them!
- If you do not get full marks on the basic tests, then your submission may not be gradable by our automated tools, and you will very likely receive a low mark for correctness.
- On the other hand, getting full marks on the basic tests does **not** guarantee your solution is correct. It only means that you spelled the name of the function correctly and passed some extremely trivial checks.
- The purpose of the basic tests is to ensure that we can grade your submission. It's possible to pass all the basic tests but still get 0% on your assignment.
- This assignment is both shorter and simpler than later assignments. If the questions on this assignment seem difficult or confusing, **come to tutorial**. [The website](#) lists some other ways you can get help.

Here are the assignment questions you need to submit.

1. **(40%): Stepping**

Mathematically there may be many possible substitutions when evaluating an expression, but as discussed in Lecture 02, for our model of computation we need to agree on exactly which one to apply in any given situation. To help reinforce the substitution rules we have an online stepper tool at:

<https://student.cs.uwaterloo.ca/~cs135/stepping/>

This tool is similar to Racket’s built-in stepper, but instead of showing you the substitution steps, it requires you to enter them.

You will need to authenticate yourself using your userid and password. Once you are logged in, you will see three sets of exercises: one on expressions, one on functions, and one on constants. Note the “Show instructions” link at the bottom of each problem. **Read the instructions!**

There are both practice questions (which you do not need to complete for marks, but should complete for your understanding) and required questions (which you do need to complete for marks for this assignment). On the practice problems, there is a “Hint” button that is enabled; use it if you get stuck. The required questions do not have a “Hint” button.

Complete the required stepping problems using the substitution rules given in lectures L01 and L02.

You can re-enter a step as many times as necessary until you get it right, so keep trying until you completely finish every question. All you have to do is complete the questions online—we will be recording your answers as you go, and there is no file to submit. The basic tests for this assignment will tell you whether or not we have a record of your completion of the stepper problems. **Note that you are not done with a question until you see the message Question complete!** You should see this once you have arrived at a final value and clicked on “simplest form” (or “Error,” depending on the question).

You are stepping through the given expressions assuming that constant definitions that appear above the expression exist, and have been fully processed as though you have pressed Run in DrRacket. **This means that you are not required to do any simplification of the constant definitions as part of the stepping.**

We realize that you can use DrRacket’s stepper to help you solve these problems, but we strongly suggest you try them on your own so that you fully understand our model of computation. In an exam situation, you will not have DrRacket’s Stepper to help you, and exams will definitely have step-by-step evaluation questions.

The Basic Tests will include a list of stepper questions that have not been completed. After completing more steppers, you’ll need to request another Basic Test to see that list updated.

2. (60%): Functions

Computers are helpful tools in many fields of human endeavour. Below are functions that are useful to compute, taken from a variety of areas.

Implement the function definitions below in Racket, using the names given. Place your solutions in the file `functions.rkt`.

Note that when you are asked to **implement** a function, it is best practice for it to be a direct implementation, though it can be any equivalent implementation. For example, when asked to implement $(a + b)$, the direct implementation is `(+ a b)`, but the implementation `(+ b a)` would be a valid equivalent implementation. When implementing x^2 , the direct implementation is `(sqr x)`, but `(* x x)` would also be a valid equivalent implementation.

For example, if we asked you to implement the function:

$$\text{mean}(x1, x2) = \frac{x1 + x2}{2}$$

you could submit:

```
(define (mean x1 x2)
  (/ (+ x1 x2) 2))
```

or:

```
(define (mean x1 x2)
  (+ (/ x1 2)
     (/ x2 2)))
```

But the one on the left is more direct. When you look at it, it's easier to see that you've implemented it correctly.

- (a) An example from geometry (*the surface area of a doughnut*):

The surface area of a doughnut that is a perfect torus depends on its minor radius r and major radius z :

$$\text{doughnut-surface-area}(r, z) = 4\pi^2 rz$$

Implement a function `doughnut-surface-area` that computes the above formula. Use the built-in value `pi`.

- (b) An example from chemistry (*the ideal gas law*): Implement a function `pressure` that computes:

$$\text{pressure}(n, T, V) = \frac{nRT}{V}$$

where n is the amount of gas in moles, T is the absolute temperature in kelvins, V is the volume in cubic metres, and R is the gas constant 8.3144626. The resulting pressure is in pascals.

- (c) An example from physics (*exponential decay of a radioactive substance*): Implement a function `q` that computes:

$$q(s, d, t) = s \cdot e^{-dt}$$

where $s \geq 0$ is the initial quantity of the substance, $d \geq 0$ is its decay constant, and $t \geq 0$ is the elapsed time.

(d) An example from music (*equal temperament*), implement a function `freq` that computes:

$$\text{freq}(\text{base-frequency}, \text{interval}) = \text{base-frequency} \cdot 2^{\text{interval}/12}$$

We might consider defining a constant `intervals-per-octave` instead of 12. While that value is very unlikely to change, the constant definition serves as an explanation. However, in this question you are asked to implement the function, not improve it.

This concludes the list of questions for you to submit solutions. Don't forget to always check the basic test results after making a submission.